

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №7
на тему
«ЭЛЕМЕНТЫ СЕТЕВОГО ПРОГРАММИРОВАНИЯ»
БГУИР 6-05-0612-02 22

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Выполнение работы	4
2.1	Архитектура системы	4
2.2	Протокол взаимодействия	4
2.3	Результат работы программы	5
	Вывод	6
	Список использованных источников	7
	Приложение А (обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является практическое освоение основ построения и функционирования сетей, стеков протоколов и программных интерфейсов. В ходе работы изучается сетевая подсистема и программный интерфейс сокетов в *Unix*-системах, а также осуществляется проектирование, реализация и отладка программ, взаимодействующих через сеть *TCP/IP*.

В рамках варианта «Удаленный командный интерпретатор» необходимо разработать:

- Агент (сервер): устанавливается на заданный порт, принимает запросы (команды), исполняет их в локальной системе и возвращает результат.
- Менеджер (клиент): соединяется с агентом, передает команды, введенные пользователем, и отображает полученный результат.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура системы

Реализованная система основана на клиент-серверной архитектуре с использованием транспортного протокола *TCP* [1]. Для обеспечения возможности одновременной работы нескольких пользователей сервер использует модель «процесс на каждое соединение» на базе системного вызова *fork* [2].

2.1.1 Серверная часть (Агент)

Сервер выполняет следующие функции:

- создание потокового сокета (*SOCK_STREAM*);
- привязку к порту, заданному аргументом командной строки;
- ожидание входящих соединений;
- для каждого нового клиента — создание дочернего процесса.

Внутри дочернего процесса реализован цикл чтения команд. Выполнение команд осуществляется через функцию *open*, которая позволяет запустить оболочку *sh*, выполнить команду и перехватить её стандартный вывод (*stdout*) и поток ошибок (*stderr*) [3]. Для уведомления клиента об окончании вывода сервер отправляет нулевой символ (*NULL-byte*).

2.1.2 Клиентская часть (Менеджер)

Клиент функционирует в интерактивном режиме:

- устанавливает соединение с сервером по указанному *IP*-адресу и порту [4];
- считывает ввод пользователя из стандартного потока ввода;
- отправляет текстовую строку команды серверу;
- ожидает и выводит данные от сервера до получения маркера завершения вывода;
- завершает работу при вводе команды *exit*.

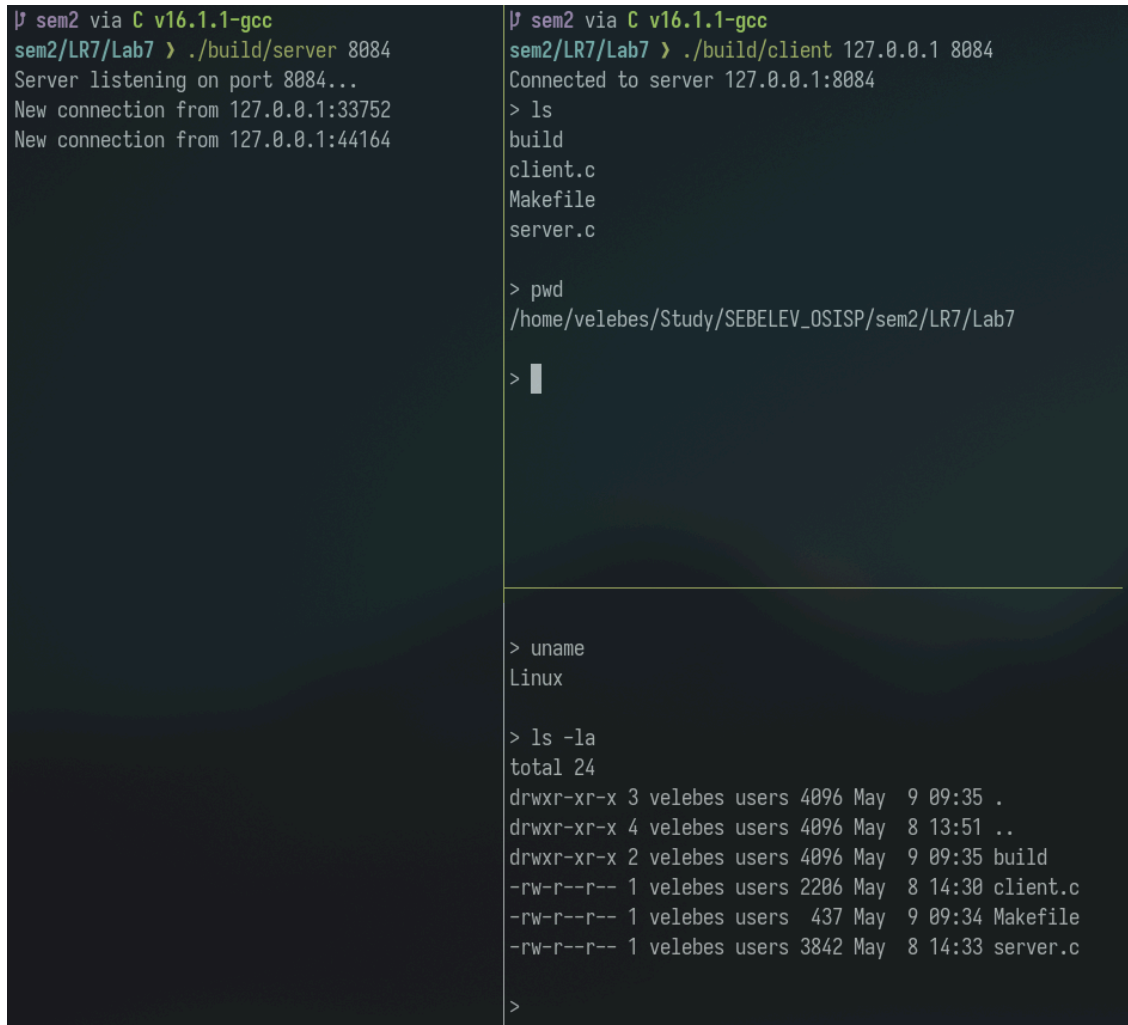
2.2 Протокол взаимодействия

Для обеспечения корректной передачи данных между клиентом и сервером был разработан простейший протокол прикладного уровня:

- 1 Клиент отправляет строку команды, завершающуюся символом конца строки.
- 2 Сервер выполняет команду с перенаправлением *stderr* в *stdout* (*2>&1*).
- 3 Сервер передает результат выполнения частями по мере готовности.
- 4 После завершения чтения из пайпа сервер отправляет байт *0x00*.
- 5 Клиент считывает данные из сокета в цикле, пока не встретит байт *0x00*.

2.3 Результат работы программы

На рисунке 2.1 представлена демонстрация работы разработанного удаленного командного интерпретатора.



```
sem2 via C v16.1.1-gcc
sem2/LR7/Lab7 > ./build/server 8084
Server listening on port 8084...
New connection from 127.0.0.1:33752
New connection from 127.0.0.1:44164

sem2 via C v16.1.1-gcc
sem2/LR7/Lab7 > ./build/client 127.0.0.1 8084
Connected to server 127.0.0.1:8084
> ls
build
client.c
Makefile
server.c

> pwd
/home/velebes/Study/SEBELEV_OSISP/sem2/LR7/Lab7

>

> uname
Linux

> ls -la
total 24
drwxr-xr-x 3 velebes users 4096 May  9 09:35 .
drwxr-xr-x 4 velebes users 4096 May  8 13:51 ..
drwxr-xr-x 2 velebes users 4096 May  9 09:35 build
-rw-r--r-- 1 velebes users 2206 May  8 14:30 client.c
-rw-r--r-- 1 velebes users  437 May  9 09:34 Makefile
-rw-r--r-- 1 velebes users 3842 May  8 14:33 server.c

>
```

Рисунок 2.1 – Пример выполнения команд через удаленный интерпретатор

В ходе тестирования была проверена работа стандартных утилит *ls*, *pwd*, *uname*, а также обработка ситуаций с вводом несуществующих команд.

ВЫВОД

В ходе лабораторной работы были изучены принципы программирования сетевых приложений в *Unix*-подобных операционных системах. Был освоен интерфейс сокетов Беркли, включая функции `socket`, `bind`, `listen`, `accept`, `connect`, `send` и `recv`. Реализованная система удаленного управления продемонстрировала особенности работы с потоковыми сокетами, необходимость обработки частичных данных при чтении из сети и важность правильного управления процессами (обработка *SIGCHLD* для предотвращения появления процессов-зомби). Полученные навыки являются фундаментальными для разработки распределенных систем и понимания работы сетевых протоколов прикладного уровня.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] TCP/IP Socket Programming in C as Client-Server Model [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/tcp-server-client-implementation-in-c/>. – Дата доступа: 09.05.2026.

[2] TCP Connections in C: A Socket Programming Guide [Электронный ресурс]. – Режим доступа: <https://dev.to/shnjd/tcp-connections-in-c-a-socket-programming-guide-k0p>. – Дата доступа: 09.05.2026.

[3] socket(7) — Linux manual page [Электронный ресурс]. – Режим доступа: <https://man7.org/linux/man-pages/man7/socket.7.html>. – Дата доступа: 09.05.2026.

[4] The Ultimate Guide to TCP Client-Server Programming in C [Электронный ресурс]. – Режим доступа: <https://idiotdeveloper.com/tcp-client-server-programming-c-guide/>. – Дата доступа: 09.05.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код файла server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>

#define MAX_CMD_LEN 1024
#define MAX_OUT_LEN 4096

void sigchld_handler(int s) {
    (void)s;
    int saved_errno = errno;
    while(waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}

void handle_client(int client_fd) {
    char cmd_buffer[MAX_CMD_LEN];
    char out_buffer[MAX_OUT_LEN];
    ssize_t bytes_received;

    while (1) {
        memset(cmd_buffer, 0, sizeof(cmd_buffer));
        bytes_received = recv(client_fd, cmd_buffer, sizeof(cmd_buffer)
- 1, 0);

        if (bytes_received <= 0) {
            break;
        }

        cmd_buffer[strcspn(cmd_buffer, "\r\n")] = 0;

        if (strlen(cmd_buffer) == 0) {
            continue;
        }

        if (strcmp(cmd_buffer, "exit") == 0) {
            break;
        }

        char cmd_with_stderr[MAX_CMD_LEN + 8];
        snprintf(cmd_with_stderr, sizeof(cmd_with_stderr), "%s 2>&1",
cmd_buffer);
```



```

FILE *fp = popen(cmd_with_stderr, "r");
if (fp == NULL) {
    const char *err_msg = "Failed to run command\n";
    send(client_fd, err_msg, strlen(err_msg), 0);
    continue;
}

int output_sent = 0;
while (fgets(out_buffer, sizeof(out_buffer), fp) != NULL) {
    send(client_fd, out_buffer, strlen(out_buffer), 0);
    output_sent = 1;
}

send(client_fd, "\\0", 1, 0);

if (!output_sent) {
    const char *no_out_msg = "(no output)\n";
    send(client_fd, no_out_msg, strlen(no_out_msg), 0);
}

pclose(fp);
}

close(client_fd);
exit(0);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    struct sigaction sa;

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt)) == -1) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);

```

```

        if (bind(server_fd, (struct sockaddr *)&server_addr,
sizeof(server_addr)) == -1) {
            perror("bind failed");
            close(server_fd);
            exit(EXIT_FAILURE);
        }

        if (listen(server_fd, 10) == -1) {
            perror("listen failed");
            close(server_fd);
            exit(EXIT_FAILURE);
        }

        sa.sa_handler = sigchld_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = SA_RESTART;
        if (sigaction(SIGCHLD, &sa, NULL) == -1) {
            perror("sigaction failed");
            exit(EXIT_FAILURE);
        }

        printf("Server listening on port %d...\n", port);

        while (1) {
            client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
&client_len);
            if (client_fd == -1) {
                perror("accept failed");
                continue;
            }

            printf("New connection from %s:%d\n",
inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

            pid_t pid = fork();
            if (pid == -1) {
                perror("fork failed");
                close(client_fd);
            } else if (pid == 0) {
                close(server_fd);
                handle_client(client_fd);
            } else {
                close(client_fd);
            }
        }

        close(server_fd);
        return 0;
}

```

Листинг А.2 – Код файла client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

```

```

#include <arpa/inet.h>
#include <sys/socket.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <IP> <Port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    const char *ip = argv[1];
    int port = atoi(argv[2]);

    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char input[BUFFER_SIZE] = {0};

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return EXIT_FAILURE;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);

    if (inet_pton(AF_INET, ip, &serv_addr.sin_addr) <= 0) {
        perror("Invalid address / Address not supported");
        close(sock);
        return EXIT_FAILURE;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr))
    < 0) {
        perror("Connection failed");
        close(sock);
        return EXIT_FAILURE;
    }

    printf("Connected to server %s:%d\n", ip, port);

    while (1) {
        printf("> ");
        fflush(stdout);

        if (fgets(input, BUFFER_SIZE, stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = 0;

        if (strlen(input) == 0) continue;

        if (send(sock, input, strlen(input), 0) < 0) {
            perror("Send failed");
            break;
        }
    }
}

```

```

    }

    if (strcmp(input, "exit") == 0) {
        printf("Exiting...\n");
        break;
    }

    memset(buffer, 0, BUFFER_SIZE);
    while (1) {
        int bytes_received = recv(sock, buffer, BUFFER_SIZE - 1, 0);
        if (bytes_received <= 0) break;

        int end_found = 0;
        if (buffer[bytes_received - 1] == '\0') {
            end_found = 1;
            bytes_received--;
        }

        if (bytes_received > 0) {
            buffer[bytes_received] = '\0';
            printf("%s", buffer);
        }

        if (end_found) break;
        memset(buffer, 0, BUFFER_SIZE);
    }
    printf("\n");
}

close(sock);
return EXIT_SUCCESS;
}

```